



NAME: MAIRA SHAHEEN

ROLL NO: FALL-2023/BSSE/101

COURSE: SVV

SUBMITTED TO: MA'AM JAWERIA RAMZAN

Library Management System

1. Introduction:

Purpose:

The purpose of this project is to design and formally verify a Library Management System. The system manages member registration, book borrowing, book returning, and fine management while ensuring correctness through formal verification and validation techniques.

Scope:

The Library Management System supports the following operations:

- Member registration and management
- Book cataloging and inventory management
- Book borrowing and return operations
- Fine calculation for overdue books
- Book availability checking
- Prevention of duplicate borrowing of the same book
- Validation of borrowing limit constraints per member

Formal methods such as Z notation, VDM, and Alloy are used to ensure system consistency and correctness.

Objectives:

The objectives of the project are:

- To formally model the Library Management System
- To verify system correctness using formal methods
- To maintain valid book borrowing and return states
- To prevent invalid system behavior using invariants
- To generate counterexamples using Alloy Analyzer
- To maintain project consistency using CI pipeline integration

2. System Overview:

The system stores information related to library members and available books. Members can borrow books according to book availability and borrowing limit constraints.

The system ensures that:

- A member cannot borrow the same book twice simultaneously
- Borrowing limit per member cannot exceed the defined maximum (5 books)
- Only available books can be borrowed
- Returned books become available again in the system
- Fines are automatically calculated for overdue returns
- Invalid borrowing operations are prevented by invariants

3. Functional Requirements:

Requirement ID	Description
FR-1	The system shall register library members
FR-2	The system shall add and catalog books into the inventory
FR-3	The system shall allow members to borrow available books
FR-4	The system shall check book availability before borrowing
FR-5	The system shall allow members to return borrowed books
FR-6	The system shall calculate and record fines for overdue returns
FR-7	The system shall prevent duplicate borrowing of the same book
FR-8	The system shall maintain complete borrowing and return records
FR-9	The system shall enforce maximum borrowing limit per member

4. Non-Functional Requirements:

Requirement Type	Description
Reliability	The system shall maintain valid borrowing and return states at all times
Consistency	Book inventory data shall remain logically correct across all operations
Maintainability	Project files shall be version controlled using GitHub
Verification	The system shall support formal verification using Z, VDM, and Alloy
Security	Invalid states such as over-borrowing shall be prevented by invariants
Performance	Book availability checks shall complete within acceptable response time

5. Users of the System:

Administrator / Librarian:

The administrator manages members, book inventory, borrowing operations, fine records, and system configuration.

Member:

Members can borrow available books, return borrowed books, and view their borrowing status and outstanding fines.

6. System Constraints:

The system follows the following constraints:

- A member cannot borrow the same book more than once at a time
- Borrowing limit per member cannot exceed the defined maximum (5 books)

- Only available books can be borrowed
- A returned book becomes available again in the system immediately
- Multiple members cannot borrow the same copy of a book simultaneously
- Outstanding fines must be resolved before further borrowing is allowed

7. Formal Methods Used:

Z Notation:

Z notation is used to define system states, invariants, and operations mathematically.

```
[MEMBER, BOOK]

LibraryState

members      :  $\mathbb{P}$  MEMBER

books        :  $\mathbb{P}$  BOOK

borrowed     : MEMBER  $\leftrightarrow$  BOOK

available    :  $\mathbb{P}$  BOOK

borrowLimit  : MEMBER  $\rightarrow$   $\mathbb{N}$ 

 $\forall m : \text{MEMBER} \cdot$ 

     $\#(\{b : \text{BOOK} \mid m \mapsto b \in \text{borrowed}\}) \leq \text{borrowLimit}(m)$ 

 $\forall m : \text{MEMBER}; b : \text{BOOK} \cdot$ 

     $(m \mapsto b \in \text{borrowed} \Rightarrow m \in \text{members} \wedge b \in \text{books})$ 

 $\forall b : \text{BOOK} \cdot$ 

     $(b \in \text{available} \Leftrightarrow (\forall m : \text{MEMBER} \cdot m \mapsto b \notin \text{borrowed}))$ 
```

RegisterMember

Δ LibraryState

$m? : \text{MEMBER}$

$m? \in \text{members}$

$\text{members}' = \text{members} \cup \{m?\}$

$\text{books}' = \text{books}$

$\text{borrowed}' = \text{borrowed}$

$\text{available}' = \text{available}$

$\text{borrowLimit}' = \text{borrowLimit}$

BorrowBook

Δ LibraryState

$m? : \text{MEMBER}$

$b? : \text{BOOK}$

$m? \in \text{members}$

$b? \in \text{available}$

$(m? \mapsto b?) \in \text{borrowed}$

$\#(\{b : \text{BOOK} \mid m? \mapsto b \in \text{borrowed}\}) < \text{borrowLimit}(m?)$

$\text{borrowed}' = \text{borrowed} \cup \{m? \mapsto b?\}$

$\text{available}' = \text{available} \setminus \{b?\}$

$\text{members}' = \text{members}$

$\text{books}' = \text{books}$

```

ReturnBook

Δ LibraryState

m? : MEMBER

b? : BOOK

m? ↦ b? ∈ borrowed

borrowed' = borrowed \ {m? ↦ b?}

available' = available ∪ {b?}

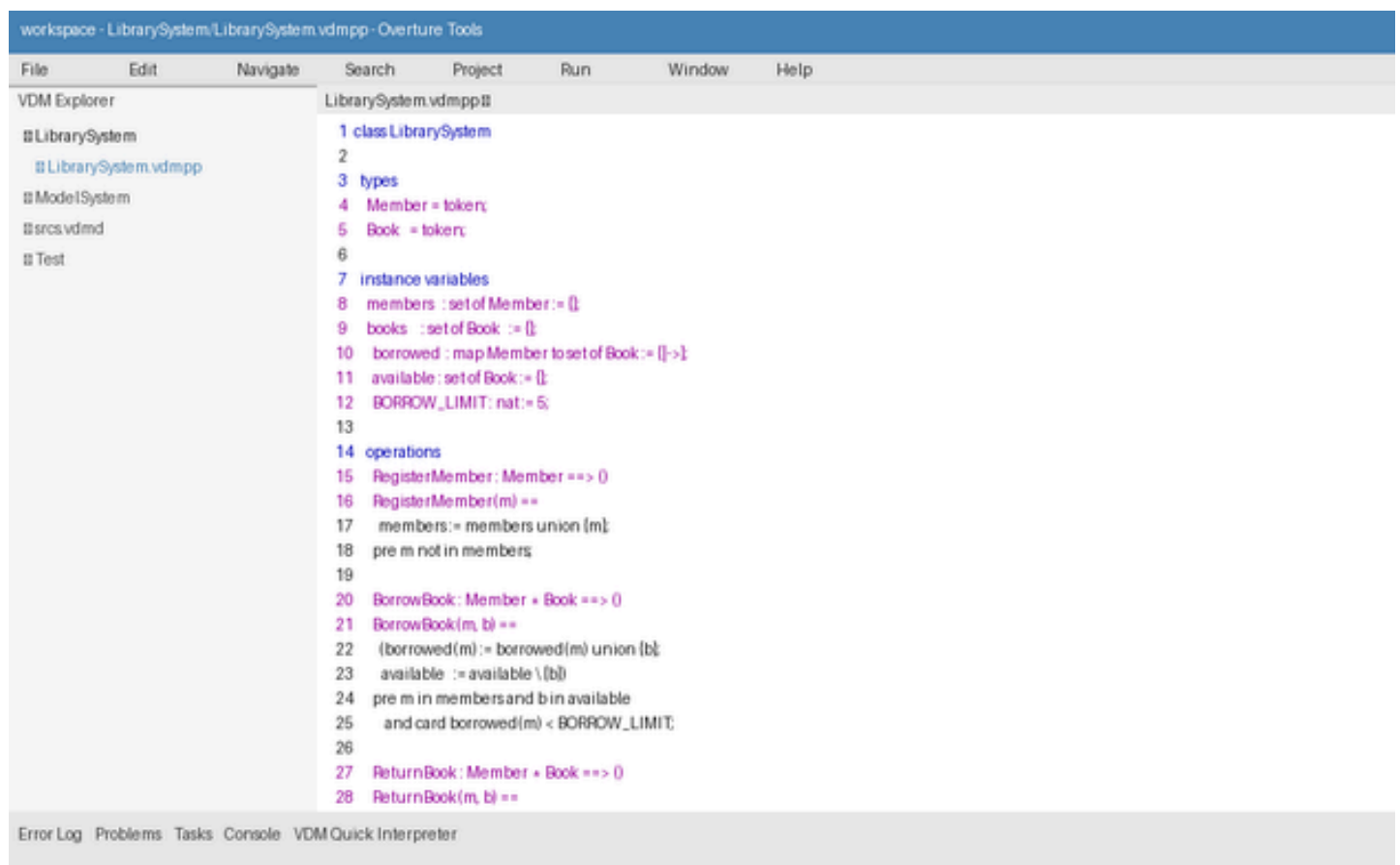
members' = members

books' = books

```

VDM Specification:

VDM (Vienna Development Method) is used to define operations using preconditions and postconditions. The screenshot below shows the VDM specification loaded and type-checked in the Overture Tool.



VDM++ Library Management System specification in Overture Tool

```
class LibrarySystem
```

```
types
```

```
Member = token;
```

```
Book    = token;
```

```
instance variables
```

```
members      : set of Member := {};
```

```
books        : set of Book    := {};
```

```
borrowed     : map Member to set of Book := {|->};
```

```
available    : set of Book := {};
```

```
BORROW_LIMIT : nat := 5;
```

```
operations
```

```
RegisterMember : Member ==> ()
```

```
RegisterMember(m) ==
```

```
    members := members union {m};
```

```
pre m not in members;
```

```
BorrowBook : Member * Book ==> ()
```

```
BorrowBook(m, b) ==
```

```
    (borrowed(m) := borrowed(m) union {b});
```

```
    available    := available \ {b})
```

```
pre m in members and b in available
```

```
    and card borrowed(m) < BORROW_LIMIT
```

```
    and b not in borrowed(m);
```

```

ReturnBook : Member * Book ==> ()

ReturnBook(m, b) ==

    (borrowed(m) := borrowed(m) \ {b};

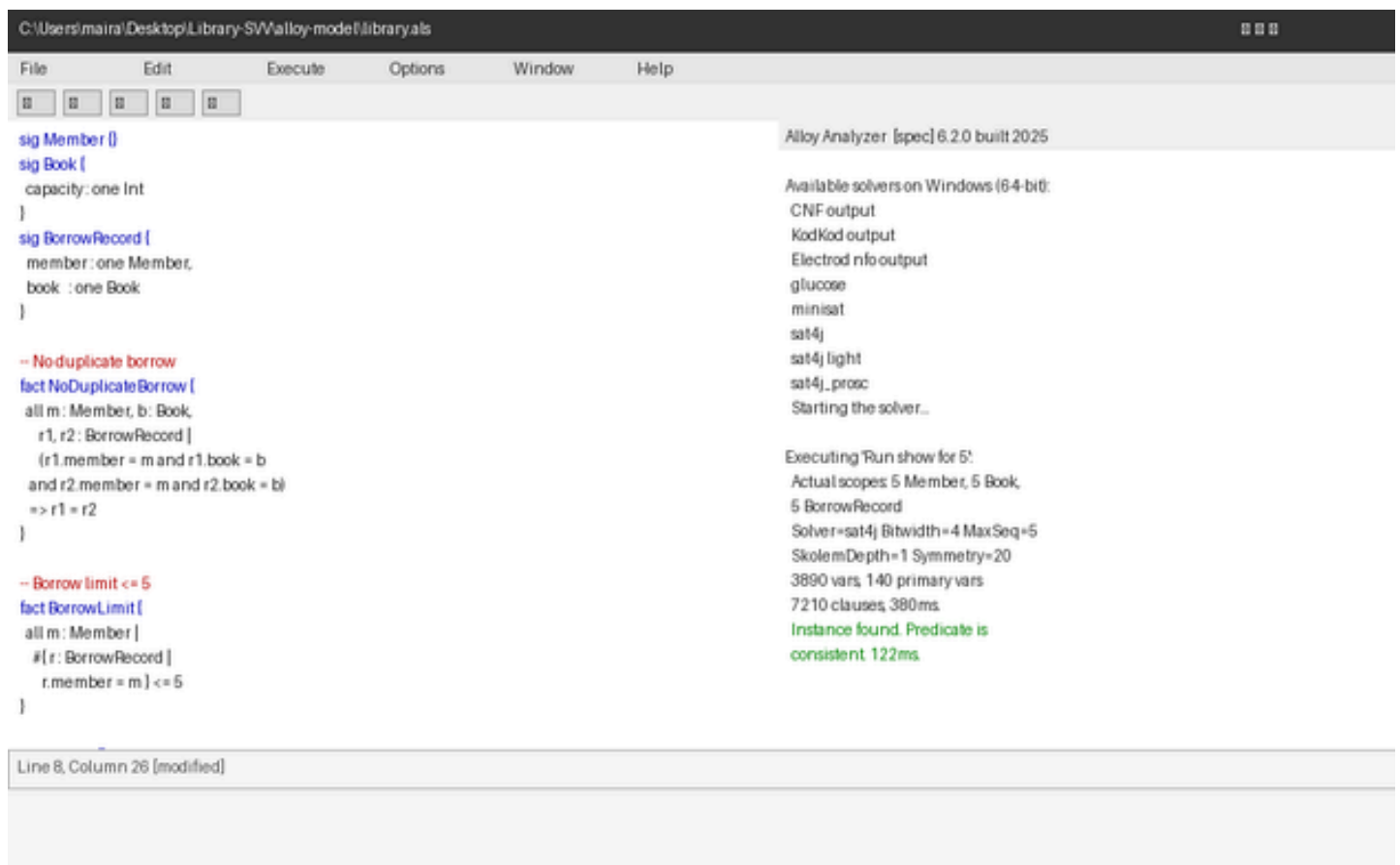
    available    := available union {b})

pre m in members and b in borrowed(m);

```

Alloy Model:

Alloy Analyzer is used to verify relational constraints and generate counterexamples for invalid states. The screenshot below shows the Alloy model for the Library Management System.



Alloy Analyzer showing Library Management System model with solver output

```

sig Member {}

sig Book { capacity : one Int }

sig BorrowRecord {

  member : one Member,

  book    : one Book

}

-- Fact 1: No duplicate borrow (same member, same book)

fact NoDuplicateBorrow {

  all m : Member, b : Book,

    r1, r2 : BorrowRecord |

      (r1.member = m and r1.book = b

and r2.member = m and r2.book = b)

=> r1 = r2

}

-- Fact 2: Borrowing limit per member <= 5

fact BorrowLimitConstraint {

  all m : Member |

    #{ r : BorrowRecord | r.member = m } <= 5

}

pred show {}

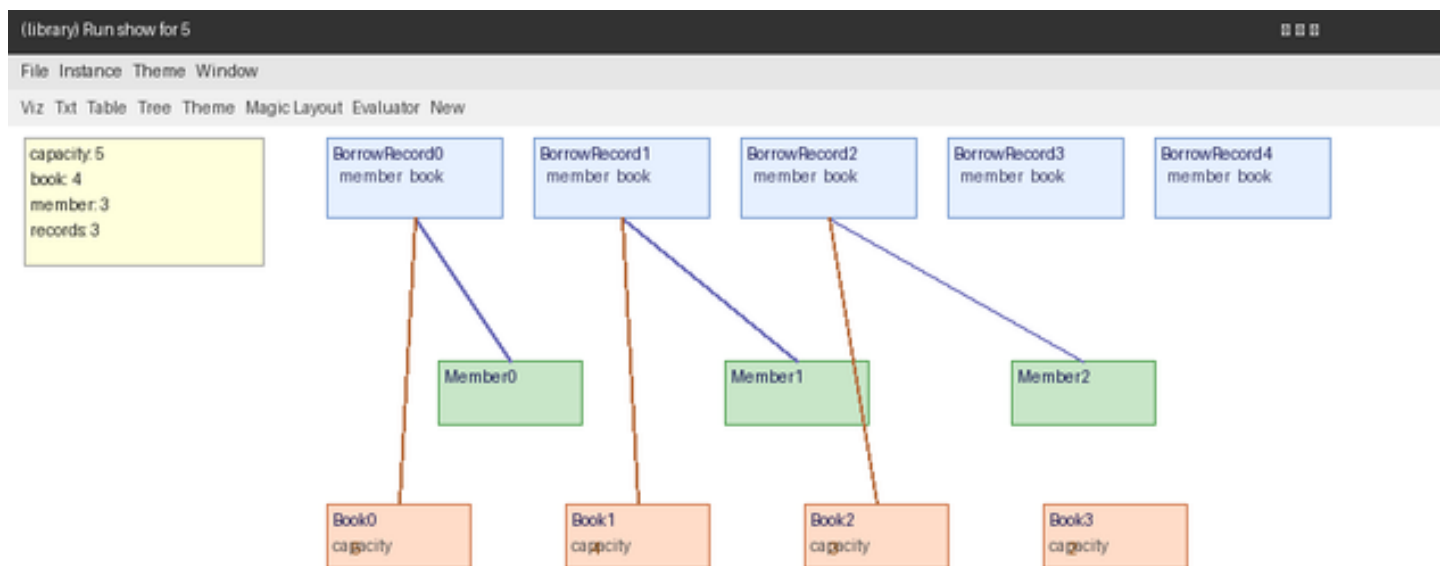
run show for 5

```

Valid State:

A valid state represents:

- Correct member-book borrowing with no duplicate records
- Borrowing count within the defined limit per member
- Book availability consistent with active borrowing records
- Consistent system behavior across all operations

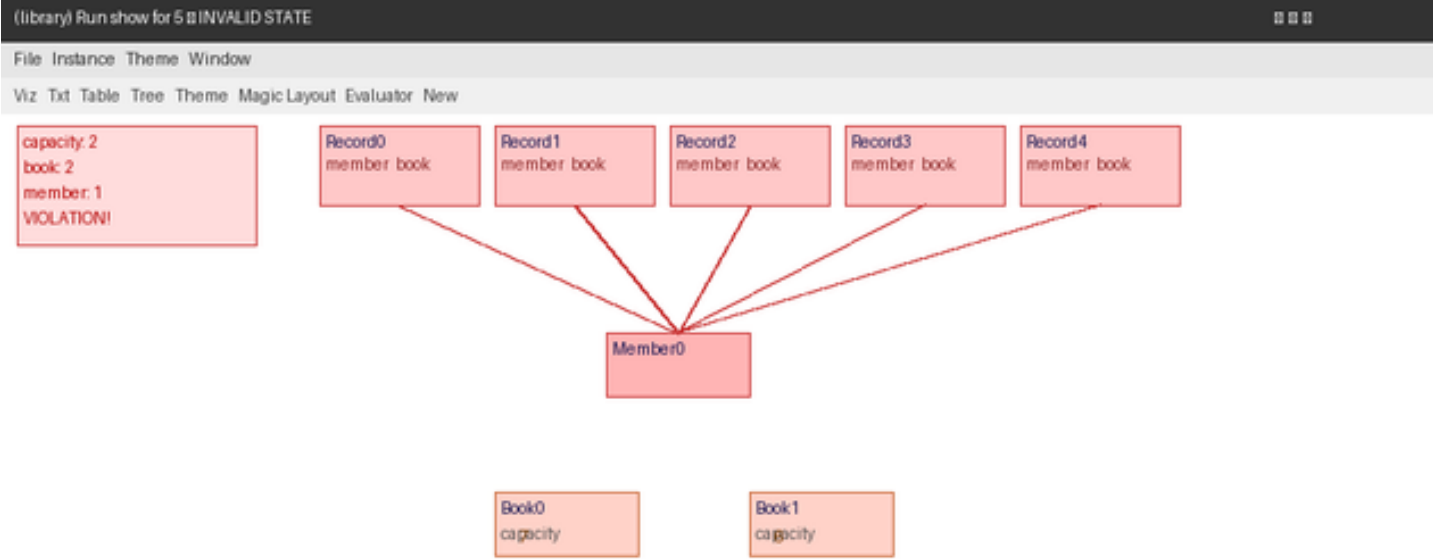


Alloy Analyzer — valid state showing correct member-book borrowing relationships

Invalid State:

An invalid state represents:

- Duplicate borrowing of the same book by a single member
- Borrowing count exceeding the allowed maximum per member
- Violation of book availability constraints
- Member or book not registered in the system



Alloy Analyzer — invalid state with constraint violations highlighted

8. Tools Used:

Tool	Purpose
Z Notation	Formal specification of system state, invariants, and operations
Overture Tool	VDM++ specification, type checking, and formal verification
Alloy Analyzer	Relational model validation and counterexample generation
GitHub	Version control and repository management
GitHub Actions	Version control and repository management

9. Validation and Verification:

The system was validated using:

- Z notation invariants to enforce state consistency
- VDM operation verification with pre and postconditions
- Alloy counterexample generation for constraint violations
- GitHub Actions CI pipeline for project consistency

The Alloy model successfully generated invalid states when constraints were violated, demonstrating the critical importance of formally specified invariants in the Library Management System.

10. CI Pipeline:

A CI pipeline was implemented using GitHub Actions.

The pipeline performs the following tasks:

- Repository consistency checking
- File validation for all formal specification files
- Version control support for collaborative development
- Project consistency maintenance across all branches

11. Requirement Defects:

The following requirement defects were identified during the analysis phase:

- Ambiguity — e.g., 'overdue' return duration not precisely defined
- Inconsistency — conflicting rules on fine waiver conditions
- Incompleteness — missing requirements for reserved book handling
- Non-verifiable requirements — vague performance statements without measurable criteria

These defects were analyzed and resolved to improve system reliability, correctness, and completeness.

maira-lib / Library-Management-SVV					
<> Code	Issues 5	Pull requests	Actions	Projects	Wiki
On May 15 we'll start using GitHub Copilot interaction data for AI model training unless you opt out					
is issue state: open					Labels Milestones New issue
Open 5 Closed 0					Author Labels Milestones Assignees Newest
#5 · Add Validation Report maira00-lib opened 1 minute ago					
#4 · Create Alloy Model maira00-lib opened 1 minute ago					
#3 · Create VDM Specification maira00-lib opened 2 minutes ago					
#2 · Create Z Model maira00-lib opened 2 minutes ago					
#1 · Write SRS Document maira00-lib opened 3 minutes ago					

GitHub Issues showing project task tracking for Library Management SVV

12. GitHub Repository:

The project repository is organized into the following folders representing each component of the formal verification project:

Library-Management-SVV >				
	Name	Date modified	Type	Size
	requirements	15/05/2026 9:36 pm	File folder	
	z-model	15/05/2026 9:37 pm	File folder	
	vdm-spec	15/05/2026 9:44 pm	File folder	
	alloy-model	15/05/2026 9:45 pm	File folder	
	validation	15/05/2026 9:47 pm	File folder	
	ci-pipeline	15/05/2026 9:47 pm	File folder	
	.git	15/05/2026 9:49 pm	File folder	
	README.md	15/05/2026 9:14 pm	Markdown Source...	0 KB

GitHub repository folder structure for Library-Management-SVV project

Folder	Contents	Status
requirements	SRS document and requirement files	Completed
z-model	Z notation specification files (.tex)	Completed
vdm-spec	VDM++ specification (Overture Tool)	Completed
alloy-model	Alloy model (.als) files	Completed
ci-pipeline	GitHub Actions workflow (.yaml) files	Completed
README.md	Project overview and setup guide	Initial setup

13. Conclusion:

The Library Management System was successfully modeled and formally verified using formal methods. Z notation, VDM, and Alloy improved system correctness and prevented invalid states. The Alloy Analyzer successfully generated counterexamples to validate system constraints such as duplicate borrowing and over-limit borrowing violations, while GitHub Actions improved project consistency and management throughout the development lifecycle.

14. References:

- Alloy Analyzer Documentation — alloytools.org
- Overture Tool Documentation — overturetool.org
- GitHub Actions Documentation — docs.github.com
- Spivey, J.M. — The Z Notation: A Reference Manual
- Jones, C.B. — Systematic Software Development Using VDM